

Security Analysis of ML-Powered Android Apps

Bruna Vasconcelos CS 6491 — Software & Systems Security, Spring 2026 University of Utah — Project Report (1-day pilot)

Abstract

On-device machine learning is increasingly embedded in mobile apps — ML SDKs are the norm in categories like keyboards, cameras, and translators (Google's *On-Device ML Practitioners Guide*, 2023; Li et al., *MLKit Usage in the Wild*, MSR '21) — yet the standard Android security tooling (FlowDroid, TaintDroid, Soot) was designed before on-device ML existed and has no awareness of ML-specific artifacts, SDKs, or data flows. This project builds a static-analysis pipeline that augments Android APK analysis with ML-aware detection: it identifies embedded model files, quantifies their exposure (via Shannon entropy and magic-byte matching), flags sensitive permissions that can feed the inference pipeline, and scans decompiled Smali for ML-SDK usage and poisoning-surface patterns. On a pilot corpus of 5 F-Droid apps (274 MB) the pipeline surfaced zero ML artifacts — a result we treat as a **pilot observation**, not a general claim about F-Droid; the more cautious reading is that F-Droid's open-source mandate likely filters out proprietary on-device ML, and the natural next step is a Play Store corpus. A synthetic positive-control APK containing a public MobileNet `.tflite` was used to validate that the detector fires when models are actually present.

1. Problem and Motivation

The Truth Serum attack (Tramèr et al., CCS 2022) poisons training data with mislabeled near-duplicates of target samples, making those samples trivially identifiable via membership inference after training. The attack presumes that the attacker can contribute training data — a presumption that maps onto any on-device ML pipeline that collects user-supplied data for retraining, personalization, or federated learning.

No prior work audits the presence or shape of those pipelines in real Android apps. FlowDroid (PLDI '14) predates widespread on-device ML and has no ML-specific sources/sinks. TaintDroid (OSDI '10) predates on-device ML entirely. Recent ML-security surveys cover lab models but not the mobile-app surface. This gap is the project's target.

2. Threat Model

- **Adversary:** a malicious training-data contributor — any party that can influence data ingested by an app's on-device training, personalization, or federated-learning loop. The

adversary does not need code execution on the device; a poisoned input delivered through the normal data-collection channel is sufficient (the Truth Serum setting).

- **Defender (this project):** an app auditor with access to a publicly-distributed APK and no access to the running app, backend services, or runtime instrumentation. Static analysis only.
- **Assets at risk:** model weights (integrity + confidentiality), training-data pipeline (integrity), user PII routed through inference.
- **Out of scope:** runtime exploits, supply-chain attacks on the build pipeline, backend-service compromises, and app repackaging / Frida (all deferred per the 1-day scope).

3. Research Questions

Primary

- **RQ1: Do poisoning surfaces exist in distributed Android apps?** Does user-submitted data flow into on-device retraining without sanitization or anomaly detection?
- **RQ2: Are model files exposed?** Are `.tflite`, `.onnx`, or similar artifacts bundled unencrypted inside APK assets with no integrity check on load?

Stretch

- RQ3: Is training data handled securely (world-readable directories, unsanitized input)?
- RQ4: Are model update channels safe (TLS, signed updates)?

4. Methodology

The analysis is static only. For each APK:

1. **SHA-256 hash** the APK bytes (reproducibility).
2. **Archive scan.* Unzip without decoding. Match file extensions in** `{.tflite, .onnx, .pb, .pt, .pkl, .h5, .mlmodel, .pth, .caffemodel}`. **For each hit, read the first 64 KB and compute Shannon entropy + magic-byte match.** **Entropy > 7.9 bits/byte is flagged as *high-entropy** — consistent with encryption or compression; the two are indistinguishable from the bytes alone.
3. **Native-library scan.** Regex over archive entries for `libtensorflowlite*.so`, `libonnxruntime*.so`, `libmlkit*.so`, `libmediapipe*.so`, `libpytorch*.so`.
4. **apktool decode** (with 300 s per-APK timeout) → Smali + `AndroidManifest.xml` + `assets/`.
5. **Manifest parse** via `defusedxml` (resists XXE/entity-expansion). Extract package name + declared permissions; flag sensitive permissions mapped to ML data sources.
6. **Smali pattern scan.** Capped at the first 5 000 Smali files per APK (files are sorted alphabetically so the scan is deterministic). When the cap truncates, a warning is recorded in the output notes. Two pattern families: ML-SDK imports (`org/tensorflow/`

lite, com/google/mlkit, ai/onnxruntime, com/google/mediapipe, org/pytorch) and poisoning-surface signatures (modelPersonalizer / PersonalizationModel / OnDeviceCustomActions, federated-learning references, feedback→retraining phrases, GradientTape / keras.fit, differentialPrivacy APIs).

7. **Severity scoring.** Critical / High / Medium / Info per finding, per the rubric in the proposal.

Outputs. Per-app JSON (full evidence, including the SHA-256 and both scanned and total Smali counts) and an aggregated `summary.csv` with one row per APK.

Intentional scope cuts for the 1-day pilot

- **No FlowDroid taint analysis.** Setup cost outweighs yield in a single-day timeline; its documented reflection / dynamic-class-loading blind spots would require manual review regardless.
- **No Frida dynamic analysis.** APK repackaging requires ~2 days per batch, per the proposal.
- **Corpus reduced from 30+ apps to 5 real apps + 1 synthetic positive control.**

5. Corpus, Ethics, and Reproducibility

Five apps from F-Droid (open-source, freely redistributable, no CFAA concern) plus one synthetic positive-control APK assembled from a public MobileNet `.tfLite` (Google Cloud storage, Apache 2.0 license).

App	Package	Size	SHA-256 (first 16)
F-Droid	org.fdroid.fdroid	12.4 MB	985f5181d48bb6ba
NewPipe	org.schabi.newpipe	10.9 MB	dbc8a1bb7a3db16f
Aves Gallery (libre)	deckers.thibault.aves.libre	55.9 MB	75697b19f2eb850f
Nextcloud Files	com.nextcloud.client	77.8 MB	e36e6ef4215cf003
Fennec (Firefox)	org.mozilla.fennec_fdroid	117.6 MB	251146a2b5f6d801
Positive control (synthetic)	edu.utah.cs6491.positive_control.ml	15.7 MB	d9c71359fcce2d5f

All analysis is performed on freely-redistributable F-Droid releases. No server interaction, no live user data, no repackaging, no PoC delivery to real devices. Responsible disclosure is framed as a 90-day window should any critical finding emerge in future Play Store runs.

6. Findings

6.1 Pilot observation: no on-device ML on the F-Droid corpus

None of the five F-Droid APKs bundle a model file, link an ML native library, or reference an ML SDK in the Smali that was scanned. Every app's top finding is an **Info-severity** "No on-device ML artifacts detected."

We treat this as a pilot observation, not a structural claim about F-Droid as a whole. Five apps is 0.1% of the F-Droid catalog, and the Smali scan is capped (Section 6.3); absence of evidence is not evidence of absence. The more defensible reading is that F-Droid's audit requirements *likely* filter out proprietary on-device ML — which would need to be confirmed on a larger corpus.

The *concrete* conclusion: F-Droid is not a productive corpus for this research question. The tool should be pointed at Play Store apps in high-risk categories (Section 7).

6.2 Positive control: detection fires as expected

The synthetic positive-control APK, containing `assets/mobilenet_v1.tflite` (16.9 MB, Google MobileNet v1), was correctly identified:

- Model artifact detected at `assets/mobilenet_v1.tflite`
- Entropy 7.51 bits/byte → flagged as *structured, not high-entropy* (correct — stock TFLite model)
- Magic bytes matched → format identified as `tflite`
- Severity: **High** ("Unencrypted model present; architecture and weights readable by anyone who can unzip the APK")

This establishes that the detector is working; the null result on the F-Droid corpus is not a tool failure.

6.3 Per-app severity distribution and Smali coverage

App	Smali scanned / total	Critical	High	Medium	Info	Top Finding
aves	5 000 / 6 692 (75%)	0	0	0	1	No on-device ML artifacts detected
fdroid	5 000 / 19 701 (25%)	0	0	0	1	No on-device ML artifacts detected
fennec	5 000 / 33 385 (15%)	0	0	0	1	No on-device ML artifacts detected
newpipe	5 000 / 8 540 (58%)	0	0	0	1	No on-device ML artifacts detected
nextcloud		0	0	0	1	

App	Smali scanned / total	Critical	High	Medium	Info	Top Finding
	5 000 / 46 547 (11%)					No on-device ML artifacts detected
pos. ctrl.	n/a (archive-only)	0	1	0	0	Model artifact at assets/mobilenet_v1.tflite

All-Info is the correct outcome when no ML artifacts are present; the tool still surfaces sensitive-permission posture (Section 6.4) and records per-APK SHA-256 for reproducibility. The **coverage column** is the important new data: the three largest apps had only 11–25% of their Smali scanned under the default 5 000-file cap. Raising the cap is a trivial config change for the Play Store pivot and is listed as the first item in Tool Limitations (Section 7).

6.4 Sensitive permissions (context)

The pipeline reported sensitive-permission posture even where no ML artifacts were present. Fennec declared the most (6), followed by Nextcloud (5), F-Droid (4), Aves (3), and NewPipe (2). These permissions are *latent* risk: if any of these apps added on-device ML in a future release, the data pipeline would already have access to CAMERA, MIC, FINE_LOCATION, or INTERNET without re-requesting consent.

7. Tool Limitations

- Smali scan cap of 5 000 files.** Nextcloud's 46 547-file Smali tree was scanned at only 11% coverage; three of five apps fell below 25%. A missed ML-SDK import in the untaken 75–89% would be a silent false negative. The cap exists to bound pilot runtime and must be raised (or replaced with priority-ordered scanning) before any claim of full coverage.
- Obfuscation blindness.** ProGuard/R8 renames `org.tensorflow.lite.Interpreter` → e.g. `a.b.c` before shipping. Pattern-based Smali detection is defeated. This is the dominant risk for the planned Play Store pivot and requires either native-library-anchored heuristics (libs are rarely obfuscated) or string-constant analysis of the resource table.
- Entropy is not a cryptographic signal.** High Shannon entropy is consistent with encryption or compression; the pipeline now labels the condition `high_entropy` (not `likely_encrypted`) and notes the ambiguity in every affected finding.
- No reflection / dynamic-class-loading analysis.** Smali pattern scan misses ML SDK use hidden behind `Class.forName` or DEX-at-runtime loaders. Same blind spot FlowDroid documents.
- Static only.** Cannot observe runtime model downloads or federated-learning rounds.
- Magic-byte signature set is not exhaustive.** Proprietary or custom model formats may evade the signature set. Entropy is a backstop.

8. Reproducibility

```
cd /home/sudosu/cs6491/project
python3 pipeline/pipeline.py --apk-dir apks/ --out results
```

Outputs land in `results/summary.csv` plus one JSON per app. Full decoded trees under `decompiled/`. Per-APK SHA-256 is recorded in both JSON and CSV. Runtime: ~3 minutes for the 274 MB corpus on a single Kali Linux laptop (Python 3.13, apktool 2.7.0, jadx 1.5.2).

9. Conclusion and Next Steps

The pilot validates two things: (a) the pipeline runs end-to-end with tools that ship on standard Linux distributions (apktool, jadx, Python + `defusedxml`) — no heroic toolchain is required; and (b) the detector fires correctly on a known-positive sample while returning a clean null on the F-Droid corpus.

The next run should target Play Store apps in three high-risk categories: keyboard apps (continuous typing data → retraining surface), translation apps (offline TFLite models), and camera/filter apps (on-device classification on PII). Two pipeline changes are needed first: raise the Smali cap (trivial) and add an obfuscation-resistant detector (moderate — anchored on native-library presence + string-constant extraction).

Combined with the Antidote defense developed in CS 6958, this tool completes the loop between lab-side defense research and field-side risk assessment: Antidote detects Truth Serum poisons in a training set; this tool identifies the real-world training sets where those poisons could be planted.

References

1. F. Tramèr et al. *Truth Serum: Poisoning Machine Learning Models to Reveal Their Secrets*. ACM CCS 2022.
2. S. Arzt et al. *FlowDroid: Precise Context, Flow, Field, Object- sensitive and Lifecycle-aware Taint Analysis for Android Apps*. PLDI 2014.
3. W. Enck et al. *TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones*. OSDI 2010.
4. B. Tran, J. Li, A. Madry. *Spectral Signatures in Backdoor Attacks*. NeurIPS 2018.
5. Google. *On-Device ML Practitioners Guide*, 2023. <https://developers.google.com/ml-kit>